

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESCUELA DE CIENCIAS Y SISTEMAS
SISTEMAS OPERATIVOS 1 SECCIÓN N
ING. SERGIO MÉNDEZ
AUX. ALLEN ROMÁN
VACACIONES DEL SEGUNDO SEMESTRE 2024



PROYECTO 2

Sistema de Registro de Notas

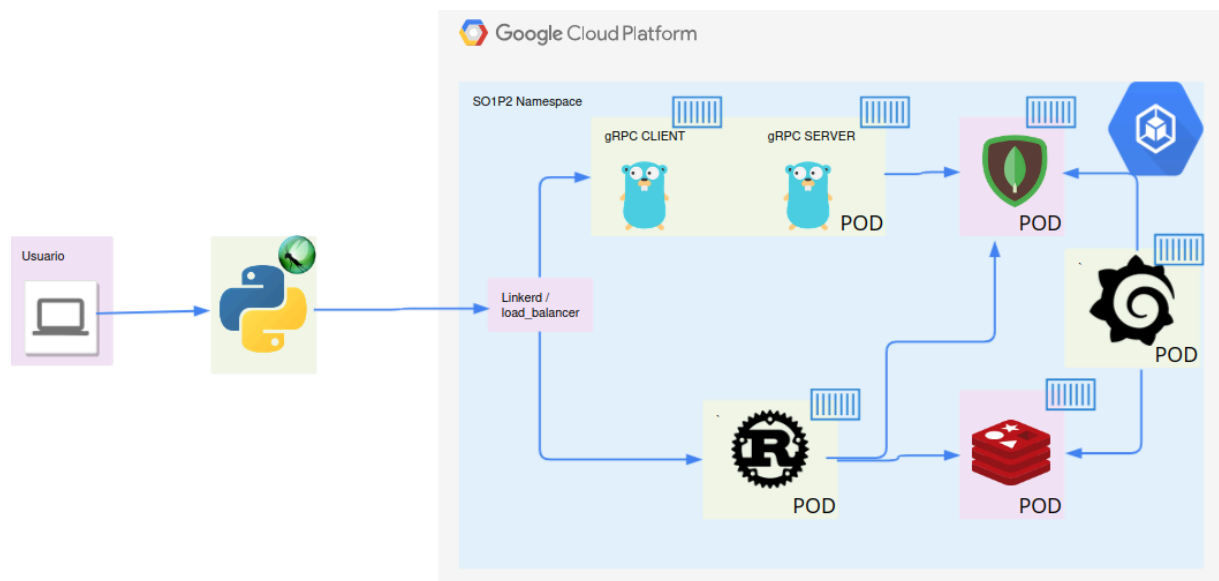
Objetivos

- Desplegar un clúster de Kubernetes y sus servicios.
- Utilizar bases de datos en la nube con MongoDB y Redis.
- Implementar balanceo de carga y observabilidad con Linkerd.
- Configurar Grafana para la visualización de datos estáticos y dinámicos.

Introducción

El objetivo de este proyecto es evaluar y mejorar el rendimiento de un sistema de ingreso de notas en línea mediante pruebas de carga y rendimiento utilizando Locust, Kubernetes y servicios en la nube como MongoDB y Redis. El sistema permitirá a los profesores o administradores académicos ingresar y gestionar las calificaciones de los estudiantes de manera eficiente. Además, se debe implementar Grafana como herramienta principal para la visualización de datos estáticos y dinámicos.

Arquitectura



Generador de Trafico

Esta parte consiste en la creación de un generador de tráfico de entrada utilizando Locust y Py como lenguaje de programación. El tráfico será recibido en un Ingress y generar los datos de prueba directamente en el script de Locust, se debe configurar Locust para que consuma un archivo de entrada almacenado como un artefacto OCI (Open Container Initiative). Este archivo contendrá los datos de prueba en formato JSON y será almacenado en un registro de contenedores, como Harbor, para su distribución.

Configuración de Locust

Se debe de generar un scripts de prueba Locust para simular el comportamiento de usuarios ingresando calificaciones. El JSON de Prueba tendrá un formato de entrada similar al siguiente:

```
{
  "carnet": 233233,
  "nombre": "Alumno 1",
  "curso": "SO1",
  "nota": 90,
  "semestre": "2S",
  "year": 2023
}
```

Los Cursos a Monitorear pueden ser:

- **SO1** - Sistemas Operativos 1
- **BD1** - Sistemas de Bases de Datos 1
- **LFP** - Lenguajes Formales y de Programación
- **SA** - Software Avanzado
- **AYD1** - Análisis y Diseño 1

La descripción para semestre puede se:

- **1S** - Primer Semestre.
- **2S** - Segundo Semestre.

Año: 2023

Nota: 50, 60, 70, 80, 90, 100.

Ingress

Se generaran dos ingress diferentes, uno para cada ruta (no usara balanceador de carga ni Linkerd).

Kubernetes

Se encargará de la orquestación de los contenedores de las diferentes partes de la aplicación. Se debe de crear un Namespace llamado: **so1p2**, y trabajar dentro de él para crear los **PODS** y los Servicios.

Rutas

En esta sección se centra en desarrollar un método altamente eficiente para registrar datos en MongoDB a través de comunicación RPC (Remote Procedure Call). El propósito es evaluar y contrastar el rendimiento con respecto a otro enfoque existente. La implementación de la primera estrategia implica recibir los datos de entrada mediante el cliente gRPC, posteriormente transmitir esta información al servidor gRPC y, por último, almacenar los datos en la base de datos de MongoDB. El flujo de la segunda ruta se inserta en Redis y también en MongoDB.

Ruta 1

La Primera Ruta será desarrollada en GOLANG, estructura:

1. grpc-client
 - a. Recibe la información y la envía al server.
2. grpc-server
 - a. Escribe en la base de datos de Mongo.

Ruta 2

La Segunda Ruta será desarrollada en Rust, estructura:

1. API REDIS:
 - a. Escribir en la base de datos de Redis.
 - b. Escribir en la base de datos de Mongo.

Autoscaling de k8s (Horizontal Pod Autoscaling)

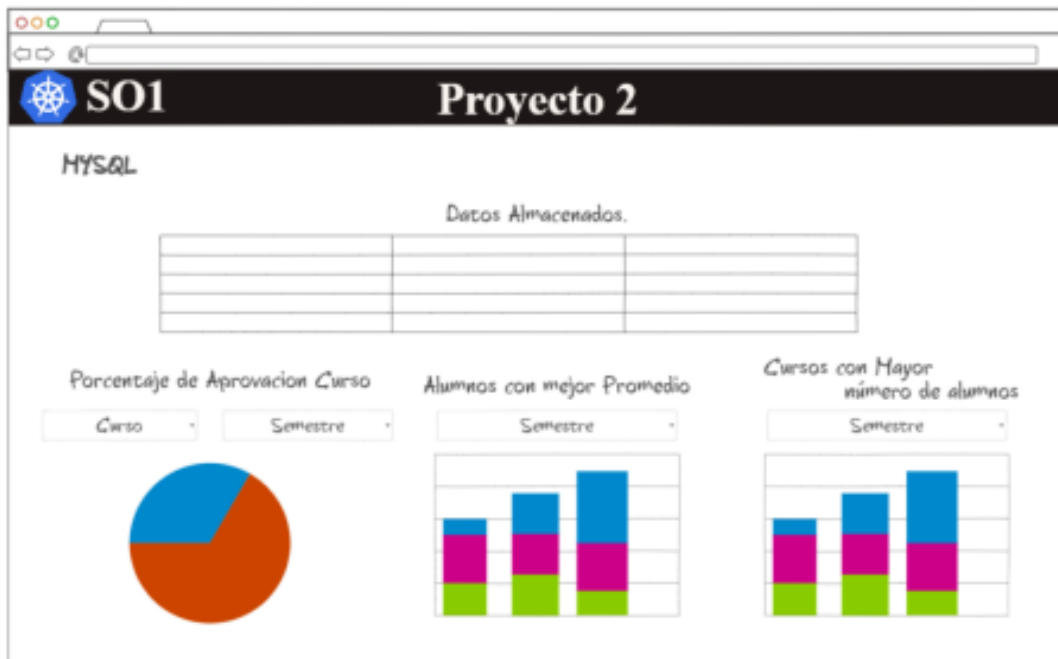
El HPA ajusta automáticamente la cantidad de PODS en ejecución en un despliegue o conjunto de réplicas según la utilización de la CPU u otras métricas. Para el Proyecto 2, cada Pod debe tener un mínimo de 1 réplica y un máximo de 3 réplicas. Además, se debe configurar que el uso de la CPU no exceda el 50%

GRAFANA

Se debe crear un sitio web para mostrar en tiempo real los datos cargados, para su desarrollo, este debe ser un Dashboard y mostrar lo siguiente:

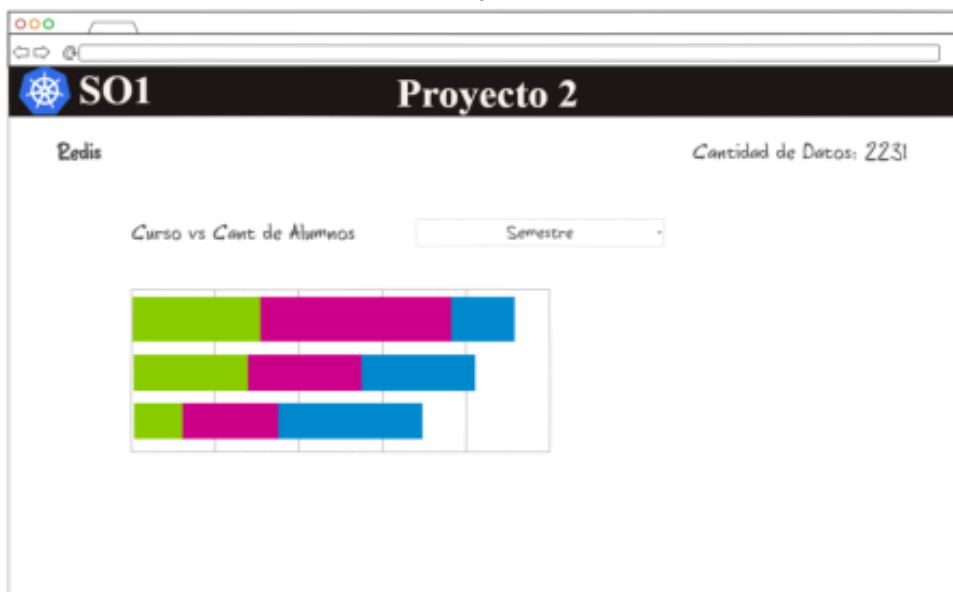
Mongo(Parte estática)

- Datos Almacenados.
- Gráfica Circular de las Notas de un Curso en un semestre. (No. Aprobados y Reprobados).
- Gráfica de Barras de Cursos con Mayor número de alumnos en un semestre específico. (Mostrar Top 3).
- Gráfica de Barras de Alumnos con mejor Promedio (Mostrar únicamente un Top 5).



REDIS (Parte dinámica)

- Cantidad Total de Registros en Tiempo Real.
- Cantidad de Alumnos en un Curso y Semestre Específico



Harbor

Harbor será utilizado como un registro de contenedores privado para almacenar las

imágenes Docker necesarias para el despliegue del sistema. Se debe configurar un proyecto dentro de Harbor con el nombre so1p2 y subir las imágenes correspondientes a los servicios desarrollados (gRPC en Golang, API en Rust, y cualquier otro componente necesario). Además, se debe asegurar que las imágenes sean accesibles desde el clúster de Kubernetes configurando las credenciales de acceso en el clúster mediante un Secret. Esto garantizará que los Pods puedan extraer las imágenes de manera segura y eficiente.

Github

- El código fuente debe de ser gestionado por un repositorio privado de Github
- Nombre de la carpeta en el repositorio de tareas: **proyecto2**
- Agregar a los auxiliares al repositorio de GitHub: **Allenrovas**

Calificación

- Al momento de la calificación se verificará la última versión publicada en el repositorio de GitHub
- Cualquier copia parcial o total tendrán nota de 0 puntos y serán reportadas al catedrático y a la Escuela de Ciencias y Sistemas.

Entregables

- **La entrega se debe realizar antes de las 23:59 del 2 de enero de 2024.**
- **La forma de entrega es mediante UEDI subiendo el enlace del repositorio.**